AI2 LAB D	Borzyszkowski Igor Album 55857 Grupa 430	**Wersja 2026-10-05**

FORMULARZE I WALIDACJA

SPIS TREŚCI

Spis treści 1
Cel zajęć 1
Rozpoczęcie 1
Uwaga 1
Prezentacja tworzenia CRUD 2
Budowa CRUD 5
Lokalizacja 6
Pomiary 8
Commit projektu do GIT 9
Podsumowanie 9

CEL ZAJĘĆ

Celem głównym zajęć jest zdobycie następujących umiejętności:

- Tworzenie typów formularzy.
- Wizualizacja formularzy w Twig.
- Obsługa formularzy w kontrolerze.
- Walidacja danych.

ROZPOCZĘCIE

Rozpoczęcie zajęć. Powtórzenie tworzenia formularzy w Symfony. Powtórzenie walidacji formularzy, grup walidacyjnych, przetwarzania formularzy (isSubmitted, isValid, handleRequest).

Wejściówka?

UWAGA

Ten dokument aktywnie wykorzystuje niestandardowe właściwości. Podobnie jak w LAB A wejdź do `Plik` -> `Informacje` -> `Właściwości` -> `Właściwości zaawansowane` -> `Niestandardowe` i zaktualizuj pola. Następnie uruchom ten dokument ponownie lub `Ctrl+A` -> `F9`.

PREZENTACJA TWORZENIA CRUD

Wykonaj komendę `make:crud` służącą do budowania kompletu operacji tworzenia, odczytu, aktualizacji i kasowania encji. Zacznij od stworzenia `LocationController` dla encji `Location`.

```
pogodynka> php .\bin\console make:crud

 The class name of the entity to create CRUD (e.g. GentleChef):
 > Location

 Choose a name for your controller class (e.g. LocationController)
[LocationController]:
 >

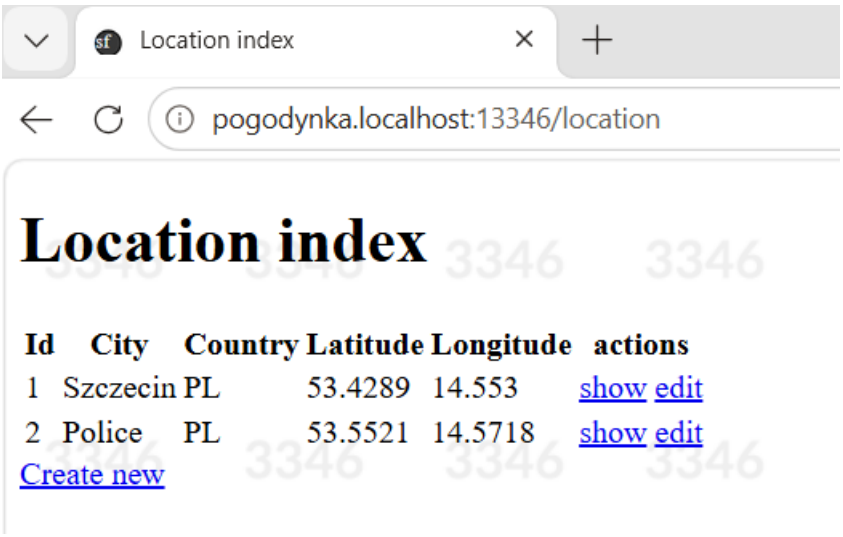
 Do you want to generate PHPUnit tests? [Experimental] (yes/no) [no]:
 >

 created: src/Controller/LocationController.php
 created: src/Form/LocationType.php
 created: templates/location/_delete_form.html.twig
 created: templates/location/_form.html.twig
 created: templates/location/edit.html.twig
 created: templates/location/index.html.twig
 created: templates/location/new.html.twig
 created: templates/location/show.html.twig


  Success!


 Next: Check your new CRUD by going to /location/
```

Zgodnie z informacją wyświetloną na końcu komendy, można od razu zobaczyć wyniki działania komendy w przeglądarce pod adresem `http://pogodynka.localhost:00000/location`:



Zbadaj zawartość:

- kontrolera `src/Controller/LocationController.php`;

- typu formularza `src/Form/LocationType.php`;
- widoków `templates/location/*`.

Zmodyfikuj `LocationType`, żeby określić typy kontrolek i ich opcje (np. wybór państwa z listy zamiast pola tekstowego). Dodajcie walidacje w dwóch grupach walidacyjnych – dla `edit` i dla `new`.

Przykłady:

```php
class LocationType extends AbstractType
{
    no usages
    public function buildForm(FormBuilderInterface $builder, array $options): void
    {
        $builder
            ->add( child: 'city',  type: null, [
                'attr' => [
                    'placeholder' => 'Enter city name',
                ],
            ])
            ->add( child: 'country',  type: ChoiceType::class, [
                'choices' => [
                    'Poland' => 'PL',
                    'Germany' => 'DE',
                    'France' => 'FR',
                    'Spain' => 'ES',
                    'Italy' => 'IT',
                    'United Kingdom' => 'GB',
                    'United States' => 'US',
                ],
            ])
            ->add( child: 'latitude',  type: NumberType::class, [
```

```php
#[Route('/new', name: 'app_location_new', methods: ['GET', 'POST'])]
public function new(Request $request, EntityManagerInterface $entityManager): Response
{
    $location = new Location();
    $form = $this->createForm( type: LocationType::class, $location, [
        'validation_groups' => 'create',
    ]);
    $form->handleRequest($request);

    if ($form->isSubmitted() && $form->isValid()) {
        $entityManager->persist($location);
        $entityManager->flush();

        return $this->redirectToRoute( route: 'app_location_index', [], status: Response::HTTP_SEE_OTHER);
    }

    return $this->render( view: 'location/new.html.twig', [
        'location' => $location,
        'form' => $form,
    ]);
}
```

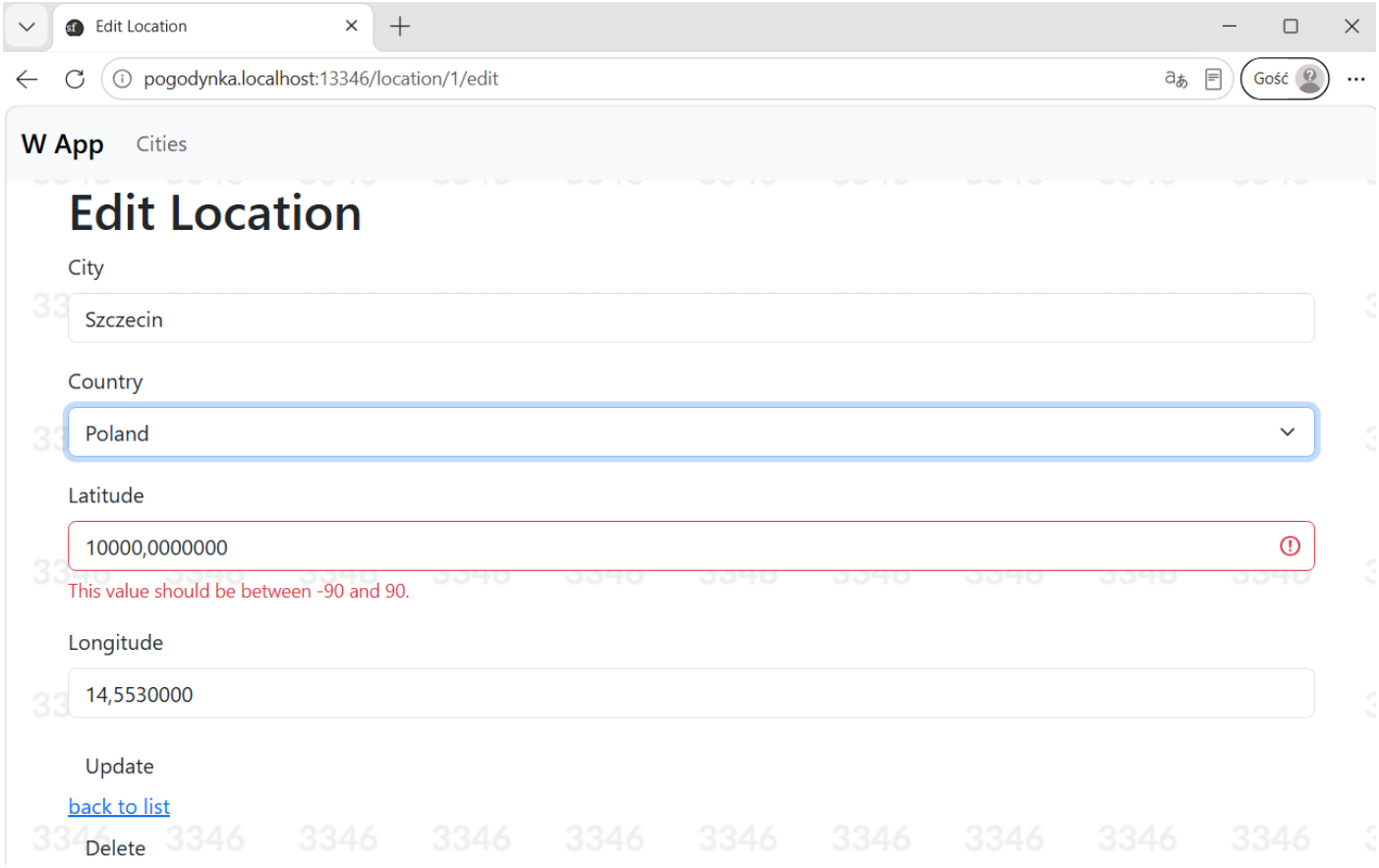
Na koniec edytuj `twig.yaml`oraz `base.html.twig` w celu zmiany renderowania strony na `Bootstrap 5`.

Asset-Mapper. Podczas edycji pliku `base.html.twig` możecie zobaczyć dziwne instrukcje `importmap`. Są one częścią pakietu asset-mapper, który domyślnie instaluje się w Symfony w wersji webapp. Mimo swojej innowacyjności, dotychczasowe doświadczenia pokazują, że podejście Webpack Encore jest po prostu prostsze i lepiej wspierane. W tym laboratorium nie będziemy używać ani Asset Mappera, ani Webpack Encore. Jeśli nie zostało to zrobione podczas poprzednich labek, to dobry moment, żeby usunąć ten pakiet, a przy okazji przetestować jak działają receptury. Po wykonaniu polecenia:

```
pogodynka> php .\composer.phar remove asset-mapper
```

linia z `importmap` zniknie z `base.html.twig`.

Przykładowy oczekiwany efekt:

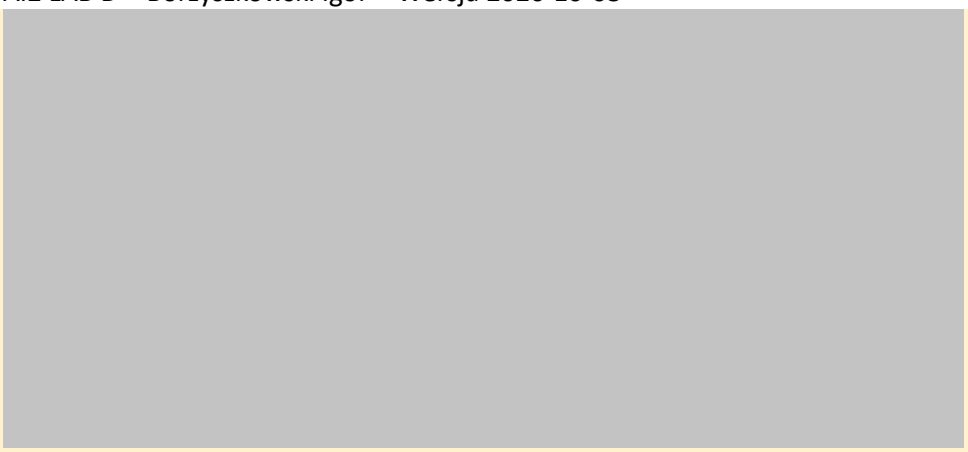


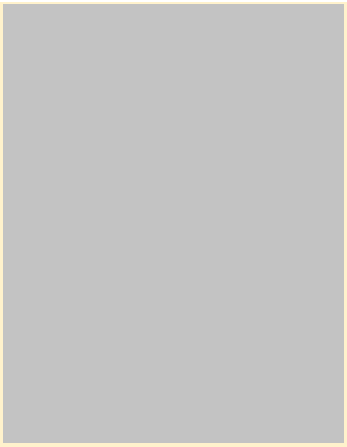
Zwróćcie uwagę, że oprócz zastosowania form_theme w Twigu oraz podłączenia styli i skryptów Bootstrapa, na powyższym zrzucie ekranu pojawił się również pasek nawigacyjny.

Budowa CRUD

Zbuduj CRUD dla encji `Measurement` podobnie jak dla `Location` oraz analogicznie formularz `MeasurementType` do tworzenia i edycji prognoz pogody, a następnie kontroler `MeasurementController` z akcjami listy, tworzenia, edycji i usuwania. Zaprezentuj osobne walidacje do tworzenia i edycji obu encji. Wykorzystaj konfigurację walidacji w YAML. Na koniec dodaj do szablonu strony (`base.html.twig`) linki do CRUDa lokalizacji i prognoz.

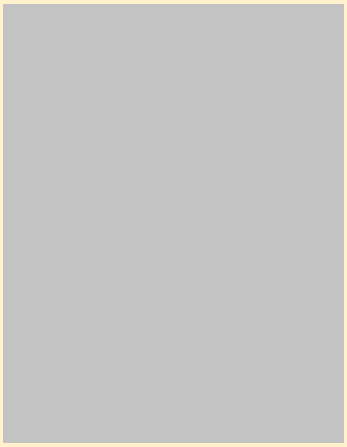
Wstaw zrzut ekranu listy lokalizacji. Upewnij się, że zastosowano layout Bootstrap 5. Wstaw zrzut wersji desktop i wersji mobilnej:





Wstaw analogiczne zrzuty dla listy pomiarów:



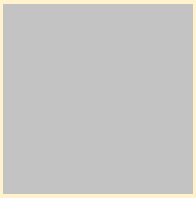


Upewnij się, że na wszystkich powyższych 4 zrzutach ekranu znajdują się linki umożliwiające przejście z lokalizacji do pomiarów i z powrotem.

Punkty:	0	1

LOKALIZACJA

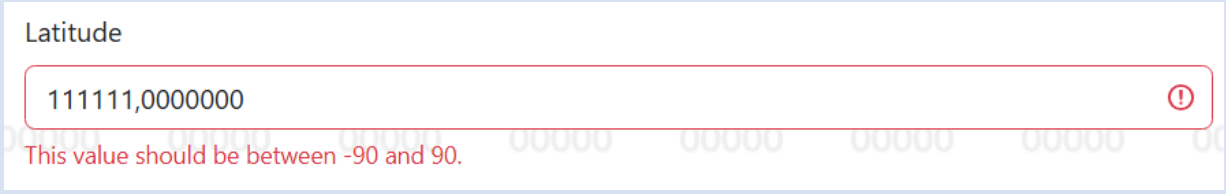
Wstaw zrzut ekranu tworzenia lokalizacji (tylko desktop). Upewnij się, że na zrzucie ekranu widoczna jest działająca walidacja realizowana przez backend (wyświetlony komunikat o błędzie w którymś polu na formularzu).



Wstaw zrzut ekranu kodu odpowiedzialnego za walidację (w formacie YAML).

Punkty:	0	1

Statystycznie 90% studentów w powyższym zadaniu popełnia błąd i robi zrzut ekranu walidacji HTML5 wykonywanej jeszcze po stronie przeglądarki, a nie walidacji wykonywanej przez backend w PHP. Poniżej przykład wyniku walidacji po stronie backendu (oczekiwany):



A tutaj przykład walidacji HTML5 (nieoczekiwany w tym sprawozdaniu):

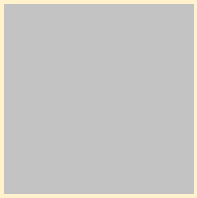


Wstaw zrzut ekranu edycji lokalizacji (tylko desktop). Upewnij się, że na zrzucie ekranu widoczna jest działająca walidacja (wyświetlony komunikat o błędzie w którymś polu na formularzu).

Wstaw zrzut ekranu kodu odpowiedzialnego za walidację (w formacie YAML).

Punkty:	0	1

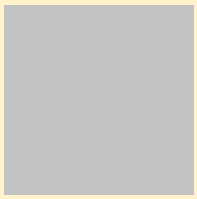
Wstaw zrzuty ekranu kasowania lokalizacji (tylko desktop).



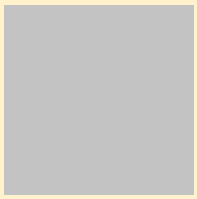
Punkty:	0	1

POMIARY

Wstaw zrzut ekranu tworzenia pomiarów (tylko desktop). Upewnij się, że na zrzucie ekranu widoczna jest działająca walidacja (wyświetlony komunikat o błędzie w którymś polu na formularzu).

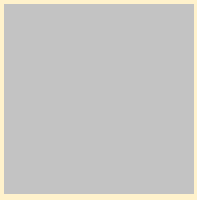


Wstaw zrzut ekranu kodu odpowiedzialnego za walidację (w formacie YAML).

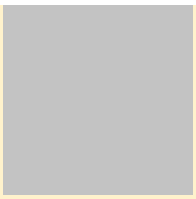


Punkty:	0	1

Wstaw zrzut ekranu edycji pomiarów (tylko desktop). Upewnij się, że na zrzucie ekranu widoczna jest działająca walidacja (wyświetlony komunikat o błędzie w którymś polu na formularzu).

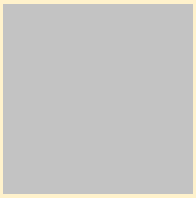


Wstaw zrzut ekranu kodu odpowiedzialnego za walidację (w formacie YAML).



Punkty:	0	1

Wstaw zrzuty ekranu kasowania pomiarów (tylko desktop).



Punkty:	0	1

COMMIT PROJEKTU DO GIT

Zacommituj zmiany. Wyślij zmiany do repozytorium (`push`). Upewnij się, czy wszystko dobrze się wysłało. Jeśli tak, to z poziomu przeglądarki utwórz branch o nazwie `lab-d` na podstawie głównej gałęzi kodu.

Podaj link do brancha `lab-d` w swoim repozytorium:

…link, np. https://github.com/ideaspot-pl/pogodynka-202610/tree/lab-d …

PODSUMOWANIE

W kilku słowach/zdaniach napisz swoje przemyślenia odnośnie tego laboratorium. Co było największym wyzwaniem? Co było najciekawsze? Co mogło być lepsze? Nie używaj LLM.

…podsumowanie…

Zweryfikuj kompletność sprawozdania. Utwórz PDF i wyślij w terminie.